LOW LEVEL DESIGN DOCUMENT

Supply Chain Management Platform

Detailed Service Design — API Contracts, Data Models, State Machines & Workflows

Version: 2.0 | **Date:** April 2026

Services: 6 Microservices + Gateway + Angular SPA

Patterns: Clean Architecture · Outbox · CQRS-Lite · MediatR Pipeline

Classification: Confidential — Internal Use Only

Table of Contents

1. Global Technical Conventions
2. Service Layer Architecture
3. Gateway Service — LLD
4. IdentityAuth Service — LLD
5. CatalogInventory Service — LLD
6. Order Service — LLD
7. LogisticsTracking Service — LLD
8. PaymentInvoice Service — LLD
9. Notification Service — LLD
10. Frontend (Angular SPA) — LLD
11. Order Lifecycle — Sequence Diagram
12. Order Status — State Machine
13. Shipment Status — State Machine
14. Entity-Relationship Diagrams
15. Outbox & Async Processing Architecture
16. Cross-Cutting Behaviors (MediatR Pipeline)
17. Error Handling Strategy
18. Testing Strategy
19. AI Exception Playbooks — API & Contract

1. Global Technical Conventions

1.1 Clean Architecture Per Service

Every microservice follows a 4-layer architecture:

API LAYER

Controllers & Auth Policies

HTTP endpoints, request routing, authorization attributes

APPLICATION LAYER

Use Cases & DTOs

Service interfaces, validators, MediatR handlers, data transfer objects

DOMAIN LAYER

Entities & Business Rules

Aggregate roots, enums, value objects, domain invariants

INFRASTRUCTURE LAYER

Persistence & Integrations

EF Core DbContext, repositories, HTTP clients, background workers

1.2 Outbox Message Contract

FIELD	TYPE	DESCRIPTION
MessageId	GUID	Unique identifier for the message
EventType	string	Event type name (e.g., OrderPlaced)
Payload	JSON text	Serialized event payload
Status	enum	Pending → Published → Failed
CreatedAtUtc	DateTime	When the event was created
PublishedAtUtc	DateTime?	When successfully published to RabbitMQ
RetryCount	int	Number of publish attempts
Error	string?	Last error message if failed

1.3 Redis Key Namespaces

NAMESPACE	PURPOSE	TTL
`idempotency:{key}`	Prevent duplicate command processing	24 hours
`revoked:{jti}`	Token revocation check	Token lifetime
`cache:{service}:{key}`	Response caching	Configurable

2. Service Layer Architecture

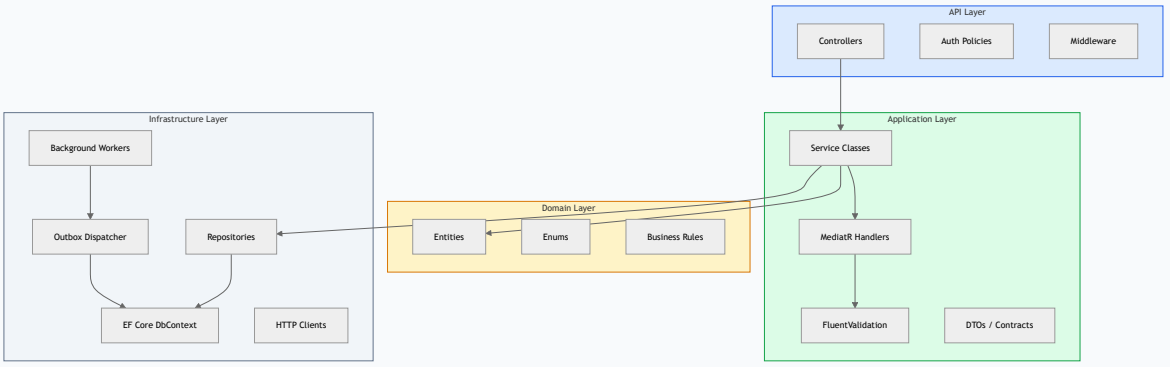


FIGURE 1 — LAYERED ARCHITECTURE PER MICROSERVICE

3. Gateway Service — LLD

3.1 Responsibilities

- Upstream entry point for all client requests (port 5000)
- Route mapping to 6 downstream microservices
- JWT authentication pass-through for protected routes
- Per-route and global rate limiting (keyed by `Oc-Client` header)
- Request correlation via `X-Correlation-Id` injection
- Round-robin load balancing across service instances

3.2 Route Mapping

UPSTREAM PREFIX	DOWNSTREAM SERVICE	AUTH	RATE LIMIT
`/identity/*`	IdentityAuth :8001	Public (auth routes) / JWT	Yes
`/catalog/*`	CatalogInventory :8002	Optional / JWT	Yes
`/orders/*`	Order :8003	JWT Required	Yes
`/logistics/*`	LogisticsTracking :8004	JWT Required	Yes
`/payments/*`	PaymentInvoice :8005	Public (credit check) / JWT	Yes
`/notifications/*`	Notification :8006	Public (ingest) / JWT	Yes

4. IdentityAuth Service — LLD

4.1 API Endpoints

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	`/api/auth/register`	No	Public	Dealer registration
POST	`/api/auth/login`	No	Public	Login & token issuance
POST	`/api/auth/refresh`	No	Public	Refresh access token via cookie
POST	`/api/auth/forgot-password`	No	Public	Initiate OTP reset flow
POST	`/api/auth/reset-password`	No	Public	Complete password reset
POST	`/api/auth/logout`	JWT	Any	Logout & token revocation
GET	`/api/users/profile`	JWT	Any	Current user profile
GET	`/api/admin/dealers`	JWT	Admin	Paginated dealer list
PUT	`/api/admin/dealers/{id}/approve`	JWT	Admin	Approve dealer
PUT	`/api/admin/dealers/{id}/reject`	JWT	Admin	Reject dealer
PUT	`/api/admin/dealers/{id}/credit-limit`	JWT	Admin	Update credit limit
GET	`/api/internal/users/{id}/contact`	API Key	Internal	Resolve user email

4.2 Data Model

ENTITY	KEY FIELDS	RELATIONSHIPS
Users	UserId, Email, PasswordHash, Role, Status	→ DealerProfile (1:1), → RefreshTokens (1:N)
DealerProfiles	DealerId, CompanyName, GSTIN, CreditLimit	→ Users (1:1)
RefreshTokens	TokenId, Token, ExpiresUtc, IsRevoked	→ Users (N:1)
OtpRecords	OtpId, Code, ExpiresUtc, IsUsed	→ Users (N:1)

4.3 Background Workers

- **IdentityOutboxDispatcher:** Polls pending outbox messages and publishes to RabbitMQ with routing key `identity.{eventType}`

5. CatalogInventory Service — LLD

5.1 API Endpoints

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	`/api/products`	JWT	Admin	Create product
PUT	`/api/products/{id}`	JWT	Admin	Update product
POST	`/api/products/{id}/restock`	JWT	Admin, Warehouse	Restock product
GET	`/api/products`	Optional	Public	Paginated product list
GET	`/api/products/{id}`	Optional	Public	Product detail
GET	`/api/products/search`	Optional	Public	Search products
GET	`/api/products/{id}/reviews`	Optional	Public/Admin	Product review list
POST	`/api/products/{id}/reviews`	JWT	Dealer	Submit product review
PUT	`/api/products/reviews/{reviewId}/approve`	JWT	Admin	Approve review (moderation)
PUT	`/api/products/reviews/{reviewId}/reject`	JWT	Admin	Reject review (moderation)
POST	`/api/inventory/soft-lock`	JWT	Admin, Dealer	Reserve stock
POST	`/api/inventory/hard-deduct`	JWT	Admin, Warehouse	Final stock deduction
POST	`/api/inventory/release-soft-lock`	JWT	Admin	Release reserved stock
POST	`/api/inventory/subscriptions`	JWT	Dealer	Subscribe low-stock alert
DELETE	`/api/inventory/subscriptions`	JWT	Dealer	Unsubscribe low-stock alert

5.2 Data Model

ENTITY	KEY FIELDS
Products	ProductId, Name, SKU (unique), UnitPrice, AvailableStock, IsActive, CategoryId
Categories	CategoryId, Name, ParentCategoryId (self-ref hierarchy)
StockTransactions	TransactionId, ProductId, QuantityChange, Type (Restock/Lock/Deduct/Release)
StockSubscriptions	SubscriptionId, ProductId, DealerId, ThresholdQty

6. Order Service — LLD

6.1 API Endpoints

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	`/api/orders`	JWT	Dealer	Create order (triggers credit check)
GET	`/api/orders/my`	JWT	Dealer	Dealer's own orders
GET	`/api/orders/{id}`	JWT	Any (ownership check)	Order detail
PUT	`/api/orders/{id}/status`	JWT	Admin, Warehouse, Logistics	Update order status
POST	`/api/orders/{id}/cancel`	JWT	Dealer, Admin	Cancel order
POST	`/api/orders/{id}/returns`	JWT	Dealer	Request return
GET	`/api/orders/{id}/saga`	JWT	Any (role-scoped)	Get saga state for order
GET	`/api/admin/orders`	JWT	Admin, Warehouse, Logistics	All orders (paginated)
GET	`/api/admin/orders/analytics`	JWT	Admin, Warehouse, Logistics	Purchase KPIs, top dealers/products
POST	`/api/admin/orders/bulk-status`	JWT	Admin, Logistics	Validate or apply bulk transition
PUT	`/api/admin/orders/{id}/approve-hold`	JWT	Admin, Warehouse	Approve credit hold
PUT	`/api/admin/orders/{id}/reject-hold`	JWT	Admin, Warehouse	Reject credit hold
PUT	`/api/admin/orders/{id}/approve-return`	JWT	Admin	Approve return request
PUT	`/api/admin/orders/{id}/reject-return`	JWT	Admin	Reject return request

6.2 Data Model

ENTITY	KEY FIELDS
Orders	OrderId, OrderNumber, DealerId, Status, TotalAmount, PlacedAtUtc, CreditHoldStatus
OrderLines	OrderLineId, OrderId, ProductId, Quantity, UnitPrice, LineTotal
OrderStatusHistory	HistoryId, OrderId, OldStatus, NewStatus, ChangedAtUtc, ChangedBy
ReturnRequests	ReturnId, OrderId, Reason, Status, RequestedAtUtc

6.3 Order Creation Flow

> **Credit Check Path:** Order → calls PaymentInvoice credit-check API → if sufficient credit → status = Processing → emit OrderPlaced event. If insufficient → status = OnHold → emit AdminApprovalRequired event → Admin can approve/reject hold.

7. LogisticsTracking Service — LLD

7.1 API Endpoints

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	`/api/logistics/shipments`	JWT	Admin, Warehouse, Logistics	Create shipment
GET	`/api/logistics/shipments/{id}`	JWT	Any	Shipment detail
GET	`/api/logistics/shipments/my`	JWT	Dealer	Dealer's shipments
PUT	`/api/logistics/shipments/{id}/assign-agent`	JWT	Admin, Logistics	Assign delivery agent
PUT	`/api/logistics/shipments/{id}/status`	JWT	Admin, Logistics, Agent	Update shipment status
POST	`/api/logistics/shipments/chatbot/ask`	JWT	Admin, Logistics, Agent	AI logistics chatbot query

7.2 Data Model

ENTITY	KEY FIELDS
Shipments	ShipmentId, OrderId, Status, AgentId, VehicleNumber, DeliveredAtUtc
ShipmentEvents	EventId, ShipmentId, Status, Timestamp, Description
ShipmentOpsStates	OpsStateId, ShipmentId, ReadinessState, RetryCount, ExceptionReason

8. PaymentInvoice Service — LLD

8.1 API Endpoints

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	`/api/payment/gateway/orders`	JWT	Dealer	Create Razorpay order
POST	`/api/payment/gateway/verify`	JWT	Dealer	Verify payment + optional capture
GET	`/api/payment/dealers/{id}/credit-check`	No	Public	Credit check by amount
PUT	`/api/payment/dealers/{id}/credit-limit`	JWT	Admin	Update credit limit
POST	`/api/payment/dealers/{id}/settlements`	JWT	Admin, Dealer	Settle outstanding balance
POST	`/api/payment/invoices`	JWT	Admin	Generate invoice
GET	`/api/payment/invoices/{id}`	JWT	Admin, Dealer	Get invoice detail
GET	`/api/payment/invoices/{id}/workflow`	JWT	Admin, Dealer	Get collection workflow state
PUT	`/api/payment/invoices/{id}/workflow`	JWT	Admin, Dealer	Upsert workflow state
GET	`/api/payment/invoices/{id}/workflow-activities`	JWT	Admin, Dealer	Activity timeline
POST	`/api/payment/invoices/{id}/workflow-activities`	JWT	Admin, Dealer	Add workflow activity
GET	`/api/payment/invoices/{id}/download`	JWT	Admin, Dealer	Download invoice PDF

8.2 Data Model

ENTITY	KEY FIELDS
DealerCreditAccounts	AccountId, DealerId, CreditLimit, UsedCredit, AvailableCredit
Invoices	InvoiceId, OrderId, DealerId, TotalAmount, GstAmount, Status
InvoiceLines	LineId, InvoiceId, ProductName, Quantity, UnitPrice, LineTotal
PaymentRecords	PaymentId, OrderId, RazorpayOrderId, RazorpayPaymentId, Status
InvoiceWorkflowStates	WorkflowId, InvoiceId, State, DueDate, ReminderCount

8.3 Razorpay Integration Flow

> **Payment Flow:** Dealer creates gateway order → receives Razorpay order ID → Frontend opens Razorpay checkout → User pays → Frontend sends payment details → Backend verifies HMAC-SHA256 signature → Optionally captures payment → Marks order as paid

9. Notification Service — LLD

9.1 API Endpoints

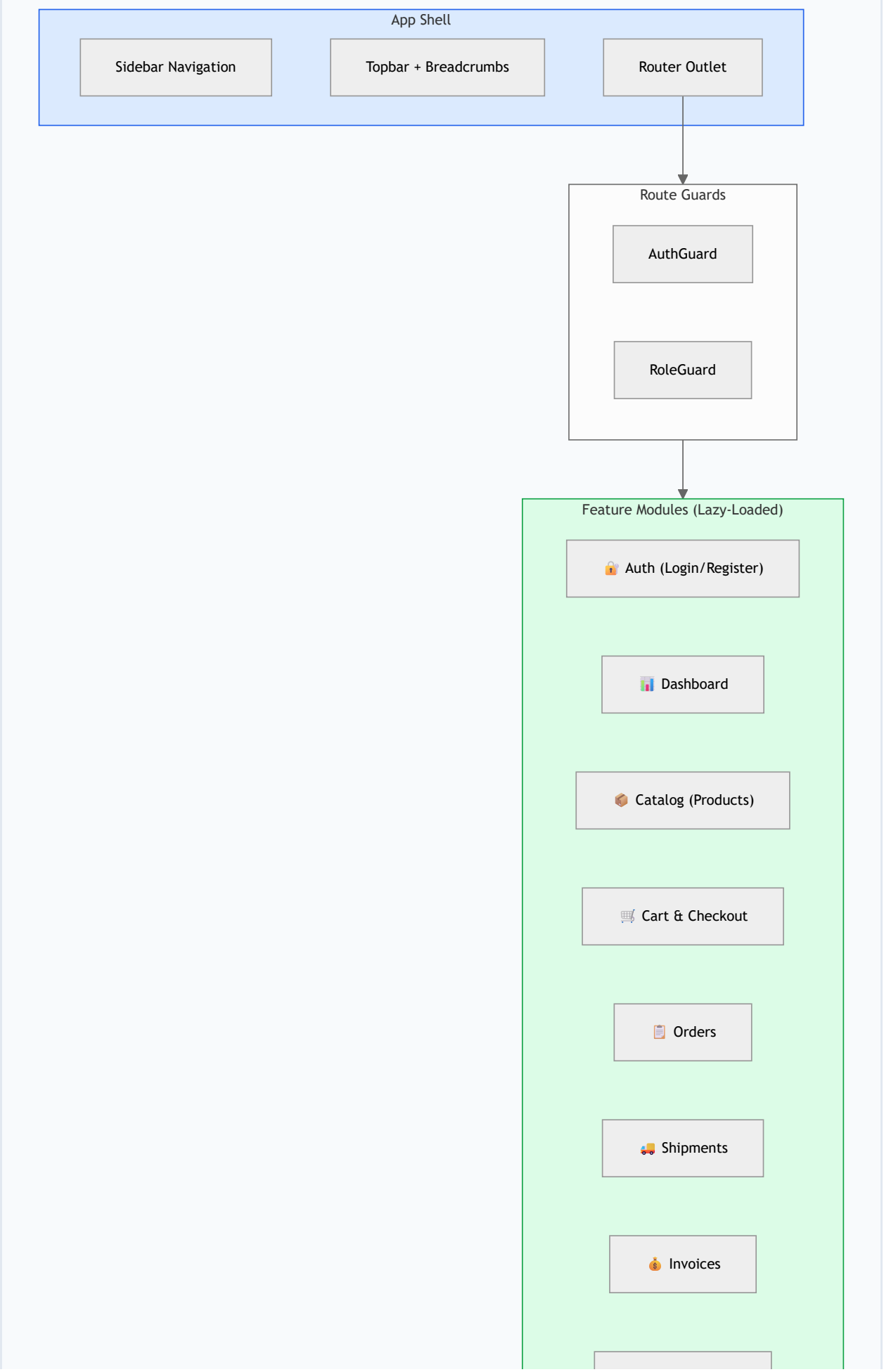
METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	`/api/notifications/manual`	JWT	Admin	Create manual notification
POST	`/api/notifications/ingest`	No	Public	Ingest integration event
GET	`/api/notifications/my`	JWT	Any	User's notifications
PUT	`/api/notifications/{id}/read`	JWT	Any	Mark as read

9.2 Background Workers

WORKER	RESPONSIBILITY	SOURCE
NotificationEventConsumer	Consumes all events from `notification.events.queue` bound to `supplychain.events` with routing key `#`	RabbitMQ
NotificationEmailDispatcher	Polls pending Email-channel notifications, resolves recipient email via IdentityAuth, sends branded HTML emails via SMTP	Database polling

10. Frontend (Angular SPA) — LLD

10.1 Architecture Overview


App Shell
Sidebar Navigation
Topbar + Breadcrumbs
Router Outlet
Route Guards
AuthGuard
RoleGuard
Feature Modules (Lazy-Loaded)
🔐 Auth (Login/Register)
📊 Dashboard
📦 Catalog (Products)
🛒 Cart & Checkout
📋 Orders
🚚 Shipments
💰 Invoices

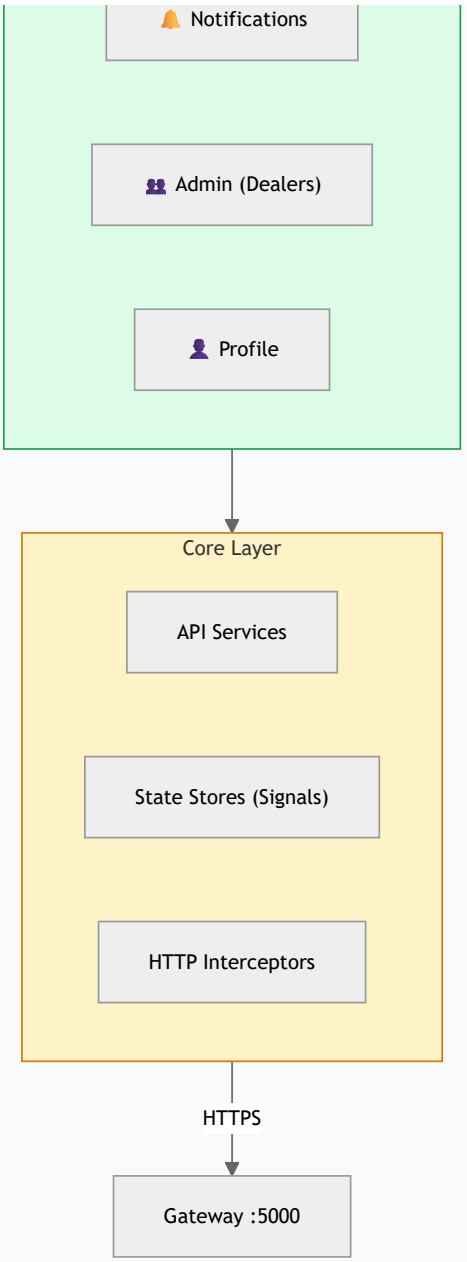


FIGURE 2 — ANGULAR SPA ARCHITECTURE

10.2 Core API Services

SERVICE	BACKEND TARGET	KEY METHODS
AuthApiService	IdentityAuth	login(), register(), refresh(), logout(), forgotPassword()
AdminApiService	IdentityAuth	getDealers(), approveDealer(), updateCreditLimit()
CatalogApiService	CatalogInventory	getProducts(), getById(), search(), createProduct()
OrderApiService	Order	createOrder(), getMyOrders(), cancelOrder()
AdminOrderApiService	Order	getAllOrders(), updateStatus(), bulkUpdateStatus()
LogisticsApiService	LogisticsTracking	getShipments(), updateStatus(), assignAgent()
PaymentApiService	PaymentInvoice	creditCheck(), createGatewayOrder(), verifyPayment()
NotificationApiService	Notification	getMyNotifications(), markRead(), markUnread()

10.3 State Management

STORE	STATE	PATTERN
AuthStore	User, Role, Token, isAuthenticated	Angular Signals + localStorage persistence
CartStore	Items, itemCount, total	Angular Signals + localStorage persistence
LoadingStore	isLoading flag	Angular Signals (used by HTTP interceptor)

11. Order Lifecycle — Sequence Diagram

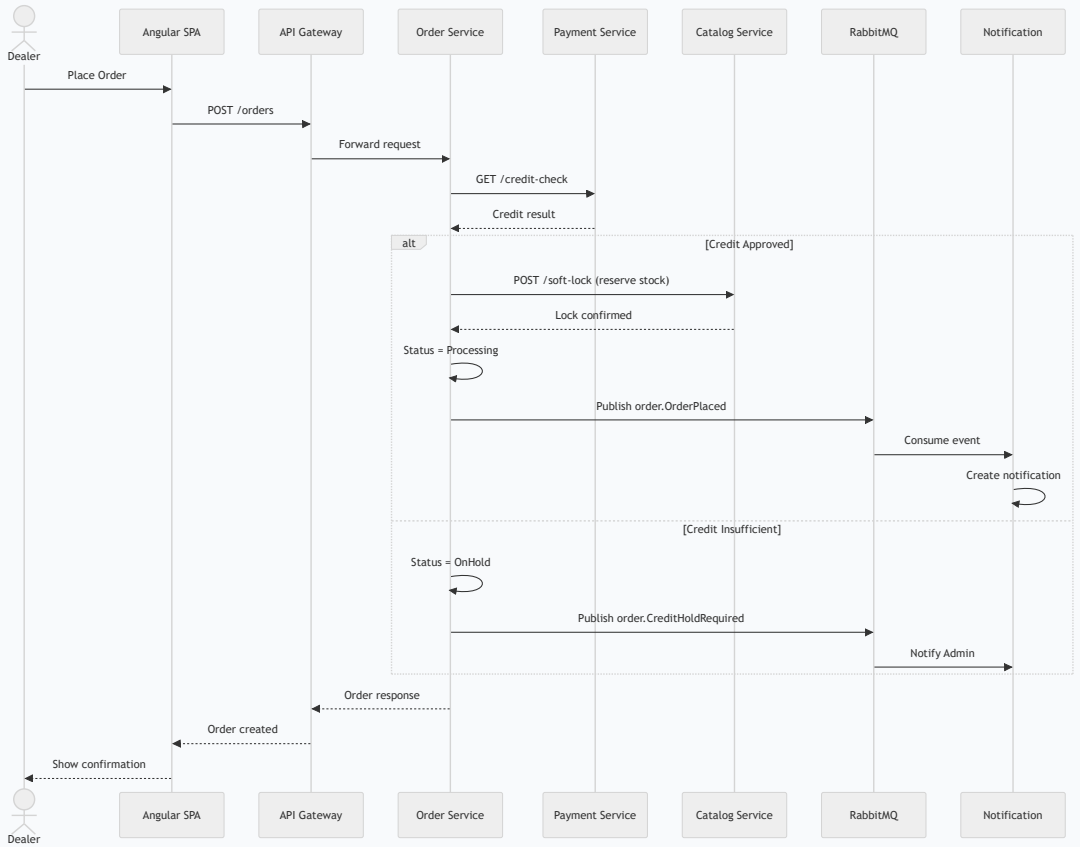


FIGURE 3 — ORDER CREATION SEQUENCE DIAGRAM (WITH CREDIT HOLD PATH)

12. Order Status — State Machine

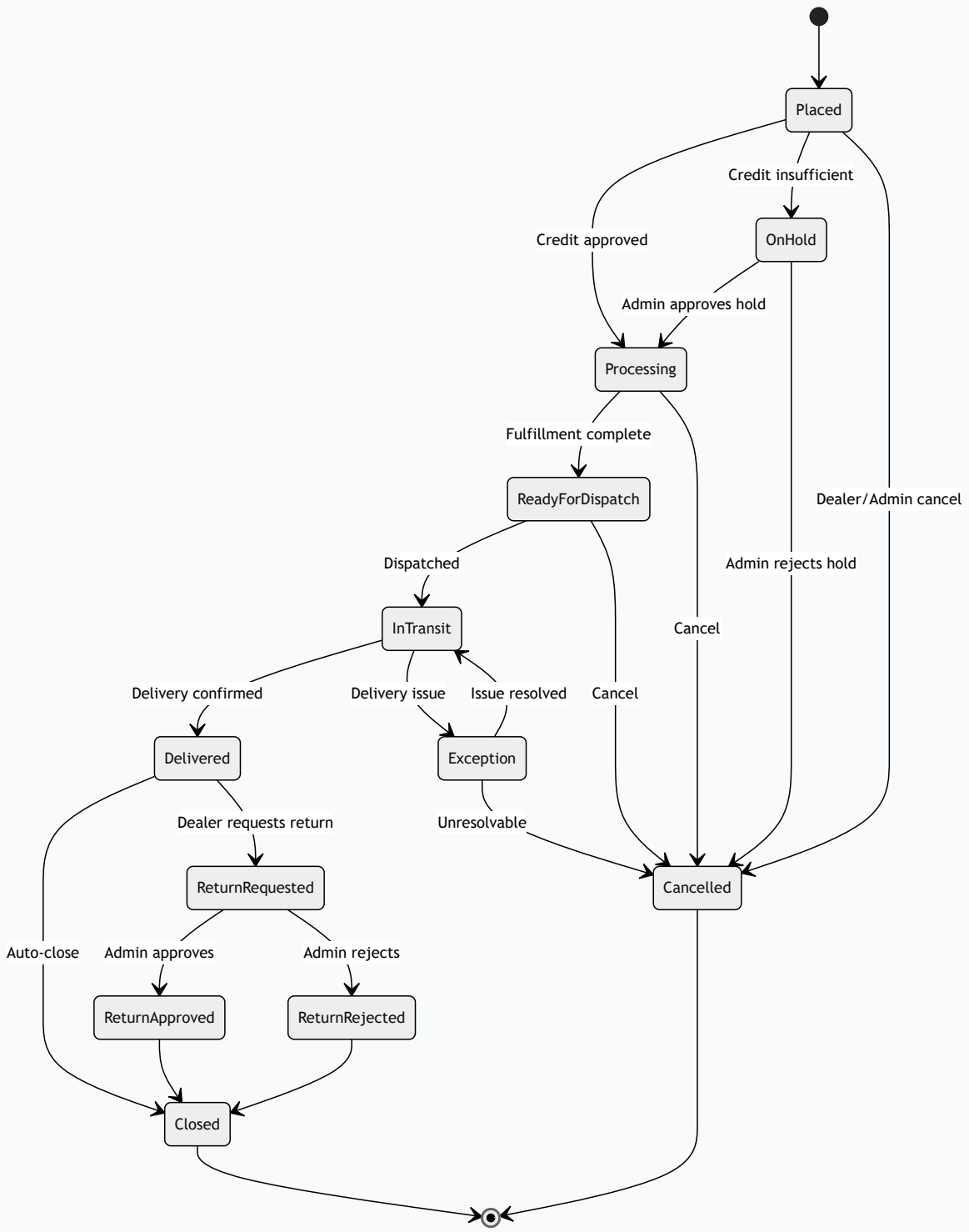


FIGURE 4 — ORDER STATUS STATE MACHINE (12 STATES, ALL VALID TRANSITIONS)

13. Shipment Status — State Machine

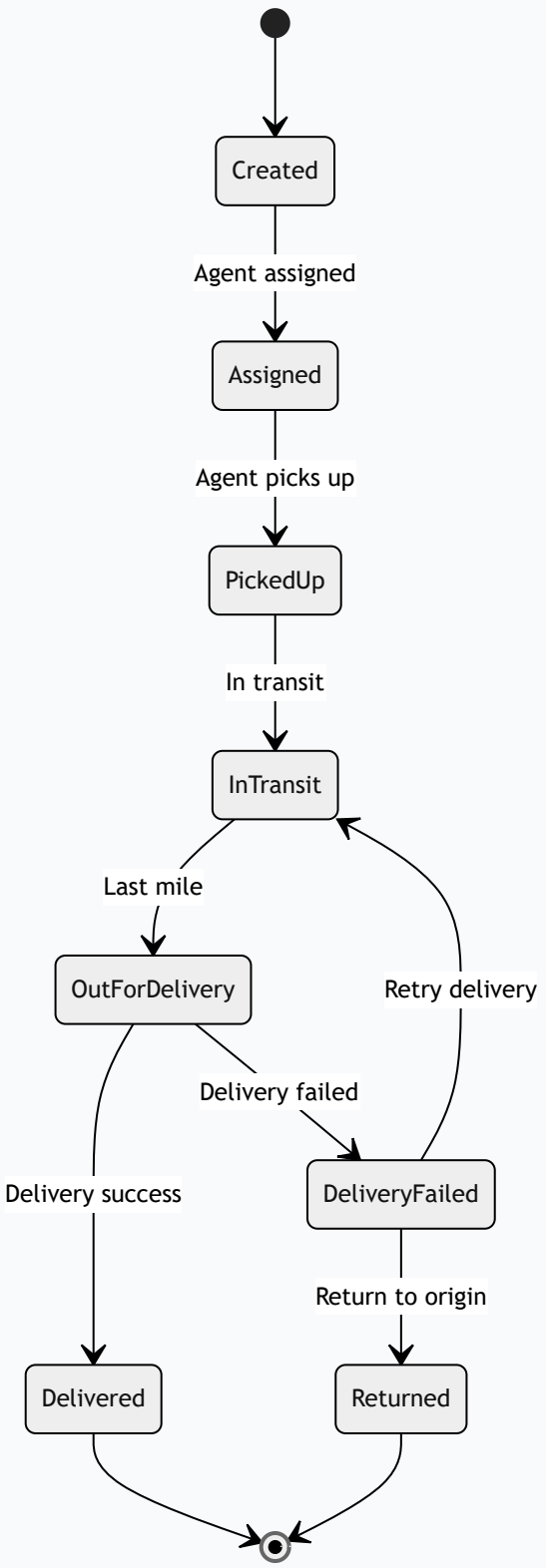


FIGURE 5 — SHIPMENT STATUS STATE MACHINE (8 STATES)

14. Entity-Relationship Diagrams

14.1 Order Domain ER Diagram

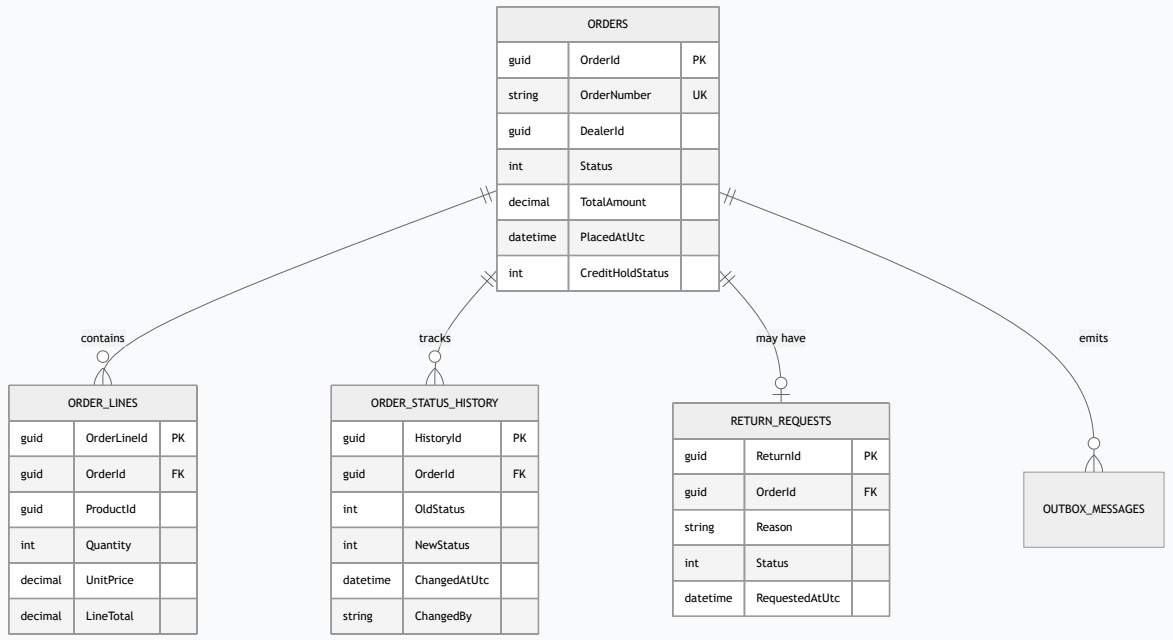


FIGURE 6 — ORDER DOMAIN ENTITY-RELATIONSHIP DIAGRAM

14.2 Payment Domain ER Diagram

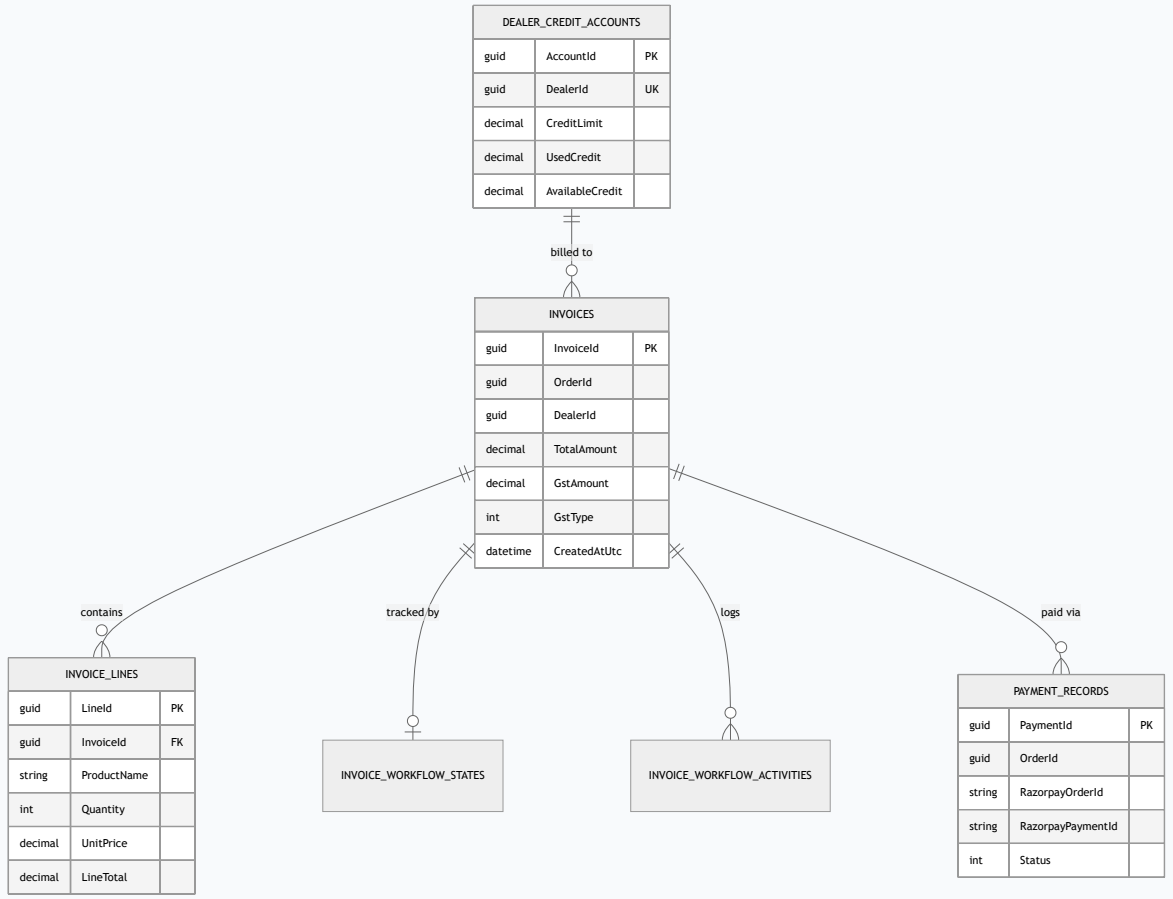


FIGURE 7 — PAYMENT & INVOICE DOMAIN ENTITY-RELATIONSHIP DIAGRAM

15. Outbox & Async Processing Architecture

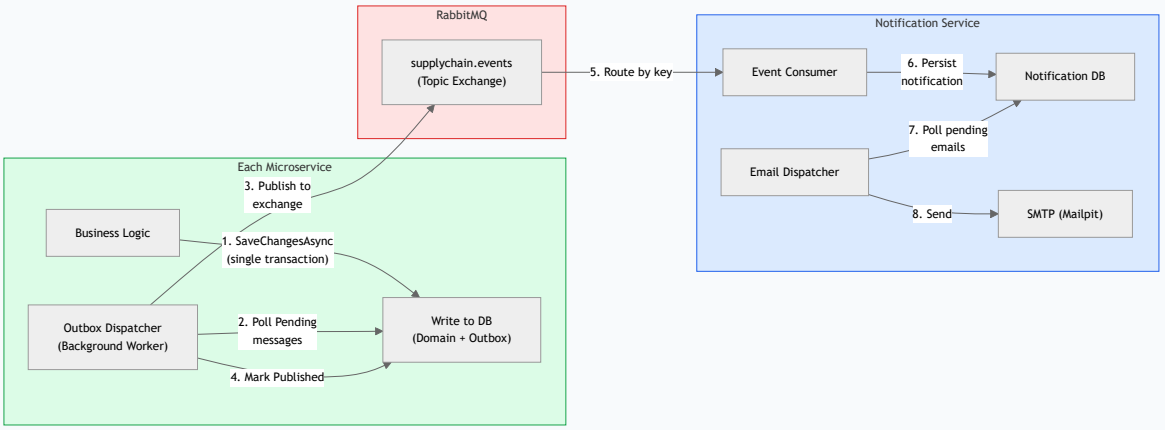


FIGURE 8 — OUTBOX PATTERN: GUARANTEED EVENT DELIVERY FLOW

> **Reliability Guarantee:** Domain data and outbox message are written in the same SQL transaction. Even if RabbitMQ is temporarily unavailable, the outbox dispatcher will retry pending messages. After max retry attempts, messages are marked as Failed for manual investigation.

16. Cross-Cutting Behaviors (MediatR Pipeline)

FIGURE 9 — MEDIATR PIPELINE: CROSS-CUTTING BEHAVIOR CHAIN

BEHAVIOR	STAGE	PURPOSE
LoggingBehavior	1st	Log request type, duration, and any exceptions via Serilog
IdempotencyBehavior	2nd	Check Redis for duplicate command key; skip if already processed
ValidationBehavior	3rd	Execute FluentValidation rules; return 400 with structured errors
TransactionBehavior	4th	Wrap handler execution in EF Core DbContext transaction

17. Error Handling Strategy

ERROR TYPE	HTTP STATUS	HANDLING
Validation failure	400 Bad Request	FluentValidation → structured property-level error response
Domain rule violation	400 Bad Request	InvalidOperationException caught by middleware
Unauthenticated	401 Unauthorized	Missing/invalid JWT token
Insufficient role	403 Forbidden	ASP.NET Core [Authorize(Roles)] policy
Resource not found	404 Not Found	Repository returns null → controller returns NotFound
Duplicate (idempotency)	200 OK (cached)	Redis idempotency key hit → return previous response
Outbox publish failure	N/A (async)	Retry with max attempts → mark Failed
Event consumer failure	N/A (async)	Redelivery → dead-letter queue after max retries

18. Testing Strategy

LAYER	TOOL	SCOPE	EXAMPLE
Unit Tests	xUnit	Domain state transitions, validation rules, workflow logic	Order.CanTransitionTo(Processing) from Placed
Integration Tests	xUnit + WebApplicationFactory	Controller auth, repository persistence, external adapters	POST /orders with Dealer JWT → 201 Created
Contract Tests	xUnit	Event payload shape, gateway route mapping	Outbox payload deserializes to OrderPlacedEvent
E2E Tests	Manual / Automated	Full lifecycle flows	Register → Approve → Order → Ship → Invoice → Pay

18.1 Test Projects

- `tests/IdentityAuth.Domain.Tests`
- `tests/CatalogInventory.Domain.Tests`
- `tests/Order.Domain.Tests`
- `tests/LogisticsTracking.Domain.Tests`
- `tests/PaymentInvoice.Domain.Tests`
- `tests/Notification.Domain.Tests`

18.2 Test Execution

```
# Run all tests with JUnit XML output
dotnet test SupplyChainPlatform.slnx \
  --logger "junit;LogFilePath=artifacts/test-results/{assembly}.xml"

# Or use the convenience script
./scripts/run-dotnet-junit-tests.ps1
```

19. AI Exception Playbooks — API & Contract

The AI Exception Playbook system provides intelligent decision support for operational exceptions (delivery failures, stock shortages, credit holds). It operates as a **human-in-the-loop** system with strict governance guardrails.

19.1 API Endpoints

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	`/api/logistics/shipments/chatbot/ask`	JWT	Admin, Logistics, Agent, Dealer	AI chatbot query for shipment insights
POST	`/api/ai/exceptions/{id}/recommend`	JWT	Admin, Logistics	Get AI recommendation for exception
POST	`/api/ai/exceptions/{id}/execute`	JWT	Admin	Execute approved recommendation
GET	`/api/ai/exceptions/{id}/history`	JWT	Admin, Logistics	Recommendation audit trail

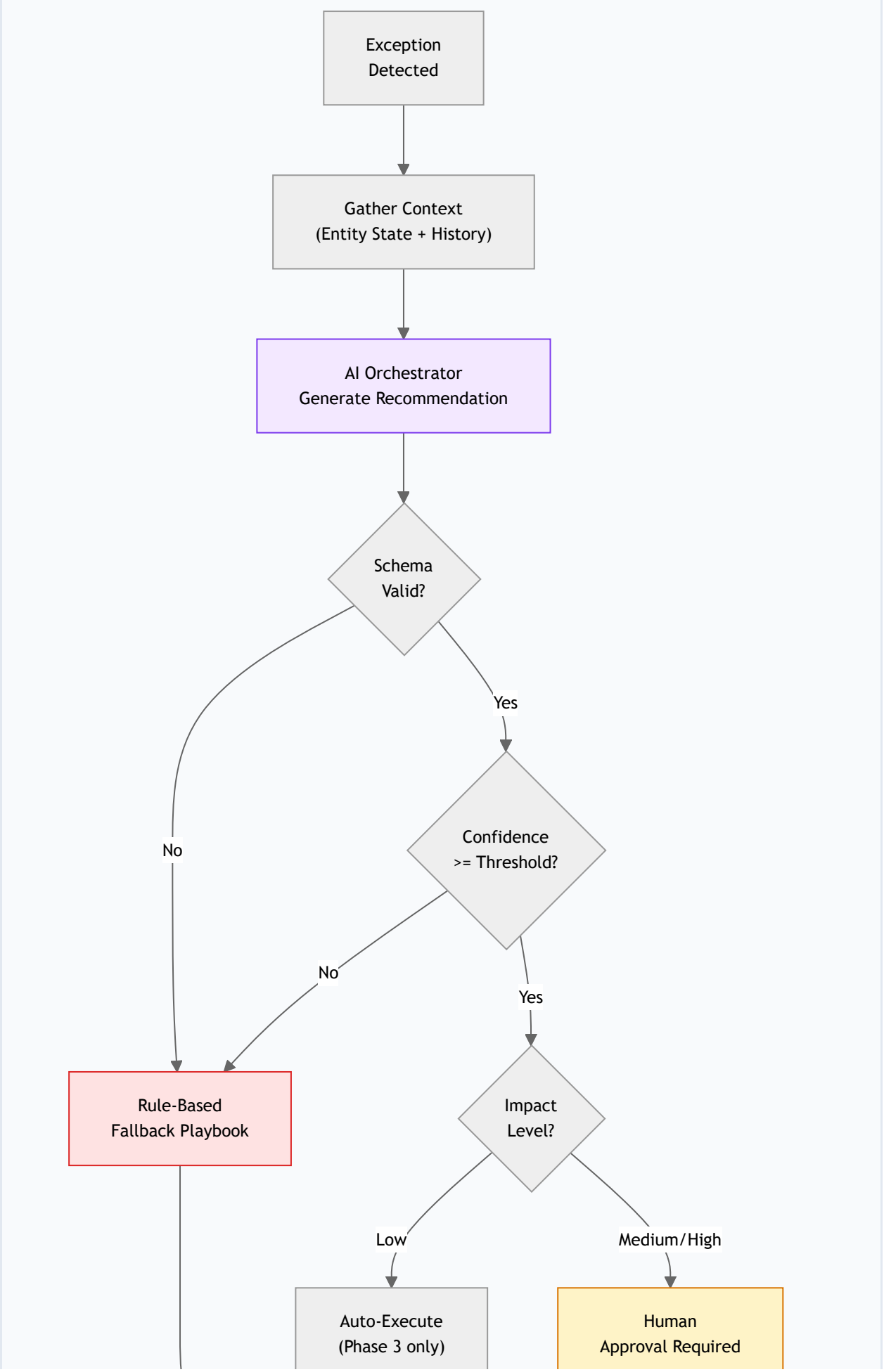
19.2 AI Recommendation Request

```
POST /api/ai/exceptions/{exceptionId}/recommend
{
  "exceptionType": "DeliveryFailed",
  "entityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "entityType": "Shipment",
  "context": {
    "currentStatus": "DeliveryFailed",
    "failureReason": "Customer not available",
    "retryCount": 2,
    "maxRetries": 3,
    "daysOverdue": 1
  }
}
```

19.3 AI Recommendation Response

```
{
  "recommendationId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "exceptionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "playbookType": "RetryDelivery",
  "suggestedActions": [
    { "step": 1, "action": "ContactCustomer",
      "params": { "channel": "SMS", "template": "delivery_reschedule" }},
    { "step": 2, "action": "RescheduleDelivery",
      "params": { "delayHours": 24, "assignSameAgent": true }},
    { "step": 3, "action": "EscalateToManager",
      "params": { "condition": "ifRetryFails" }}
  ],
  "confidenceScore": 0.87,
  "expectedImpact": {
    "etaReductionMinutes": 120,
    "serviceRiskBefore": "High",
    "serviceRiskAfter": "Low",
    "costDeltaEstimate": -450.00
  },
  "explanationText": "Customer was unavailable during previous 2 attempts...",
  "requiresHumanApproval": true,
  "modelVersion": "gpt-4o-2024-05-13",
  "generatedAtUtc": "2026-04-14T10:30:00Z"
}
```

19.4 Decision Flow


Exception Detected
Gather Context (Entity State + History)
AI Orchestrator Generate Recommendation
Schema Valid?
Yes
No
Confidence >= Threshold?
No
Yes
Rule-Based Fallback Playbook
Impact Level?
Low
Medium/High
Auto-Execute (Phase 3 only)
Human Approval Required

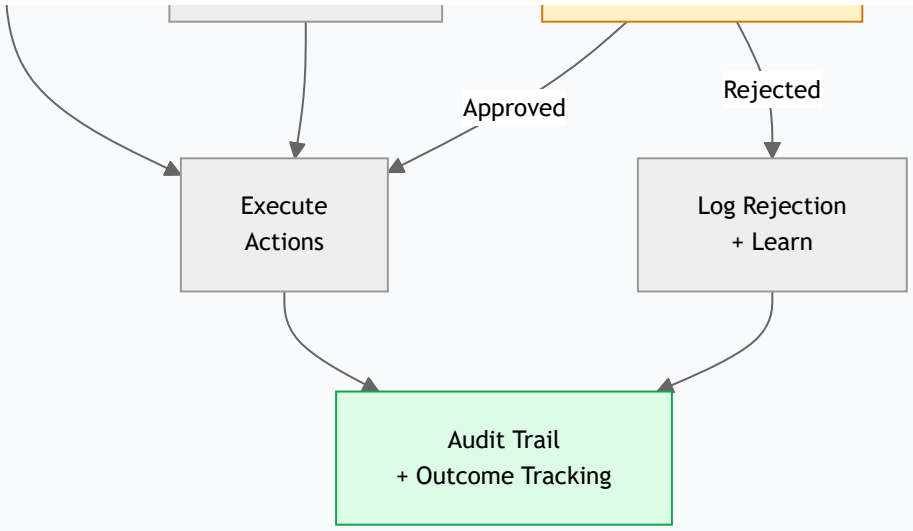


FIGURE 10 — AI EXCEPTION RESOLUTION DECISION FLOW

19.5 Playbook Catalog

PLAYBOOK TYPE	TRIGGER	TYPICAL ACTIONS
RetryDelivery	Shipment delivery failed	Contact customer → Reschedule → Escalate
RerouteShipment	Agent unavailable / vehicle breakdown	Reassign agent → Update ETA → Notify dealer
StockShortage	Inventory below threshold	Alert warehouse → Suggest restock qty → Hold affected orders
CreditHoldResolution	Order on credit hold	Check payment history → Suggest approval/rejection → Notify dealer
InvoiceOverdue	Invoice past due date	Send reminder → Escalate to admin → Suspend credit

Supply Chain Management Platform — Low Level Design Document v2.0
Confidential · April 2026 · Page generated for print/PDF export